

Comparison of XGBoost and Deep Neural networks

1. The following is the result of my execution of XGBoost and Neural Networks

Method used	Dataset size	Training-set (accuracy)	Validation- set (accuracy)	Test -set (accuracy)	Time taken for the model to be fit (seconds)
XGBoost in Python via scikit-learn and 5-fold CV	1000	100	92.75	92.00	0.20
	10000	98.22	95.40	95.75	0.41
	100000	96.36	95.91	95.93	1.83
DNN with 1 hidden layer 4 nodes	1000	97.66	94.38	92.00	38.52
	10000	96.48	96.62	95.95	150.39
	100000	96.62	96.83	96.40	875.31
DNN with 2 hidden layers of 4 nodes each	1000	96.41	94.37	96.00	15.74
	10000	96.44	96.50	96.65	51.10
	100000	96.22	96.48	96.51	439.24

2. Based on the result of Deep Neural Network, I would say that more numbers of hidden layers has a significant impact on the execution time here, the reason could be due to the *poor parallelization* while using 1-layer model and costs long time for convergence due to overhead, surprisingly small matrix operation does not use hardware efficiently. [{source}](#)
3. Based on the result that we have achieved, I can establish that XGBoost is superior to deep neural networks in terms of execution time and predictions. Which is quite obvious, as deep neural networks excel while working with large datasets and mostly multi-dimensional matrix data objects.